Tutorial: Calculating RMU reliability for a go/no-go task

Giacomo Bignardi

2025-09-11

Introduction

> **Note**
>
> This tutorial can be run and tested inside the docker image `bignardig/tidyverse461:v6`, which has all the required R packages pre-installed. The `v6` image differs from the `v5` image used for the paper's analysis scripts only in that it ships the newer version of `gbtoolbox`, in which the `reliability()` function has been renamed `rmu()`. The old name still works (with a deprecation warning), so the analysis code is unaffected.
>
> You don't need docker to follow along, though – you can also run the code the usual way, in your own R installation. The code in Installing the required R packages will install all the necessary R packages for you.

This is a brief walk-through of how to calculate reliability using the approach (relative measurement uncertainty; RMU) described in our pre-print.

In this example, we will calculate RMU reliability for the `d'` parameter estimated from a go/no-go task. We will also compare our results to other methods, including test-retest, split-half and empirical reliability.

For a more detailed tutorial and background on signal detection theory models, I'd recommend the first few chapters of Detection Theory: A User's Guide and Matti Vuorre's Excellent Blog, which will explain how these models work in more detail.

In this tutorial, I'm going to assume that you're familiar with the basics of Bayesian modelling and brms, and focus on the reliability methods.

Getting the code and data

The source file for this tutorial (`tutorial_calculating_rmu_gonogo.qmd`) and the go/no-go data it uses (in `data/osf_hedge_cwzds/`) are both in the paper's GitHub repository, giac01/8_bayes_reliability. The easiest way to get everything is to clone the repository, which needs git to be installed (the download is around 120 MB, as the repository also contains the paper's simulation code and results):

```
git clone https://github.com/giac01/8_bayes_reliability.git
cd 8_bayes_reliability
```

If you'd rather not use git, you can download the same files as a zip archive from the repository page (the green **Code** button, then **Download ZIP**, or this direct link) and unzip it.

The tutorial `.qmd` files sit in the root folder of the repository, and the data paths in the code below are relative to that folder. If you open the `.qmd` file in RStudio and run the chunks or render it, the working directory is set to the repository root for you. If you're instead running the code in a plain R session, set the working directory to the repository root first, e.g. `setwd("~/8_bayes_reliability")`.

If you're using the docker image, start the container from inside the cloned folder, mounting it as the working directory. For example, the following renders this tutorial:

```
docker run --rm -it --user rstudio \
  -v "$PWD":/home/rstudio/8_bayes_reliability \
  -w /home/rstudio/8_bayes_reliability \
  bignardig/tidyverse461:v6 \
  quarto render tutorial_calculating_rmu_gonogo.qmd
```

Installing the required R packages

The code below checks for the R packages this tutorial needs and installs any that are missing. All of them are on CRAN, except for my package of miscellaneous R functions, `gbtoolbox`, which contains the `rmu()` function used to calculate RMU. `gbtoolbox` is installed from github; if you already have an older version installed, which has `reliability()` but no `rmu()`, it will be updated.

```
# Install any missing CRAN packages (does nothing if they are all already
installed)
required_packages = c("tidyverse", "tidybayes", "brms", "gt", "ggdist", "knitr",
"remotes")
missing_packages           =      required_packages[!sapply(required_packages,
requireNamespace, quietly = TRUE)]
if (length(missing_packages) > 0) install.packages(missing_packages)

# Install (or update) gbtoolbox from github if rmu() isn't available
if   (!requireNamespace("gbtoolbox",   quietly   =   TRUE)   ||   !"rmu"   %in%
getNamespaceExports("gbtoolbox")) {
  remotes::install_github("giac01/gbtoolbox")
}
```

! Installing Stan (`rstan` or `cmdstanr`)

`brms` fits models using Stan, which needs to be installed separately from the R packages above. Two R interfaces ("backends") to Stan are available:

- **`rstan`** is `brms`'s default backend, and is what this tutorial uses. It is installed from CRAN automatically alongside `brms`, but needs a working C++ toolchain to compile models – see the rstan installation guide if `brm()` fails with a compilation error.
- **`cmdstanr`** is usually faster and is not on CRAN. Install the R package from the Stan R-universe repository, then install `cmdstan` itself:

```
install.packages("cmdstanr", repos = c("https://stan-dev.r-universe.dev",
getOption("repos")))
cmdstanr::check_cmdstan_toolchain(fix = TRUE)
cmdstanr::install_cmdstan()
```

See the cmdstanr installation guide for details. To use it, add `backend = "cmdstanr"` to the `brm()` calls below.

Load Data

We'll use test-retest data from Hedge, Powell & Sumner's reliability paradox paper, pooling data from studies 1 and 2, which contain different participants. Retest data was collected three weeks after initial testing. Data from the paper is openly available, downloaded from here.

Reference: Hedge, C., Powell, G., & Sumner, P. (2018). The reliability paradox: Why robust cognitive tasks do not produce reliable individual differences. Behavior research methods, 50, 1166-1186.

```
rm(list=ls(all.names = T))
set.seed(10)
library(tidyverse)
library(tidybayes)
library(brms)
library(gt)
library(gbtoolbox)

path_to_data = file.path("data","osf_hedge_cwzds")
data_files    = list.files(path_to_data, all.files = TRUE, recursive = TRUE,
pattern = ".csv$", full.names = TRUE )

data_list    = lapply(data_files, function(x)
  read.csv(
    file = x,
```

```
    header = FALSE,
    col.names = c("block","trial","stimulus","condition","correct","rt")
    )
  )

study_numbers        <- as.integer(sub(".*Study(\\d+)_P(\\d+)gng(\\d+)\\.csv",
"\\1", data_files))
participant_numbers  <-  as.integer(sub(".*Study(\\d+)_P(\\d+)gng(\\d+)\\.csv",
"\\2", data_files))
session_numbers      <- as.integer(sub(".*Study(\\d+)_P(\\d+)gng(\\d+)\\.csv",
"\\3", data_files))

for (i in seq_along(data_files)){
  data_list[[i]]$study   = study_numbers[i]
  data_list[[i]]$session = session_numbers[i]
  # Participant numbers restart from 1 in study 2, so add 1000 to keep IDs unique
across studies
   if (study_numbers[i] == 2) {participant_numbers[i] = participant_numbers[i]
+ 1000}
  data_list[[i]]$pps     = participant_numbers[i]
}

data_long = do.call("bind_rows", data_list)

data_long = data_long |>
  arrange(pps, session)

data_long |>
  slice(1:15) |>
  knitr::kable(
    caption = "This is what our data looks like"
  )
```

block	trial	stimulus	condition	correct	rt	study	session	pps
1	1	4	0	1	0.82081	1	1	1
1	2	4	0	1	0.46026	1	1	1
1	3	4	0	1	0.34787	1	1	1
1	4	4	0	1	0.36472	1	1	1
1	5	4	0	1	0.33133	1	1	1
1	6	4	0	1	0.32326	1	1	1

block	trial	stimulus	condition	correct	rt	study	session	pps
1	7	4	0	1	0.34668	1	1	1
1	8	4	0	1	0.29894	1	1	1
1	9	4	0	1	0.25250	1	1	1
1	10	4	0	1	0.26738	1	1	1
1	11	4	0	1	0.28376	1	1	1
1	12	4	0	1	0.28424	1	1	1
1	13	4	0	1	0.43602	1	1	1
1	14	4	0	1	0.30716	1	1	1
1	15	4	0	1	0.25173	1	1	1

Go/No-Go Task and Signal Detection Theory

In this go/no-go task, one of four letters is shown on a screen. Participants are told to press the space bar as quickly as possible if three of the four letters appear on screen ("go" condition trials), and to not press the space bar if one of the letters appears on screen ("no-go" condition trials).

In the data, the `condition` column is coded `0` for go trials (450 per session) and `2` for no-go trials (150 per session), and `correct` is `1` if the participant responded correctly on that trial (pressed the space bar on a go trial, or withheld a response on a no-go trial) and `0` otherwise.

	Response	
Condition	Go	No-Go
Go	Hits	Misses
No-Go	False alarms	Correct rejections

We will analyse the go/no-go task using a signal detection theory model to derive a measure of performance called sensitivity or `d'`.

The number of "go" responses, that is, trials when the participant presses the space bar, will be the outcome variable. Condition ("go" or "no-go") will be our predictor. Participants who are better on the task should have a high probability of responding "go" in the go condition, and a low probability of responding "go" in the no-go condition.

In this example, one could calculate `d'` directly using each participant's hit rate (HR_i; proportion of go responses in the go condition) and false alarm rate (FAR_i; proportion of go responses in the no-go condition), shown in the equation below. In fact, this gives us estimates of `d'` that are largely identical to the Bayesian method (see Appendix A), and one could calculate reliability utilizing the split-half method (see Split-Half Reliability). However, this approach won't work for more complex models, which is where RMU is particularly helpful.

$$d'_i = \Phi^{-1}(HR_i) - \Phi^{-1}(FAR_i) \quad (1)$$

Note that Φ^{-1} represents the inverse cumulative distribution function of the standard normal distribution (in R, this function is `qnorm()`). It converts values between 0-1 to z-scores.

To estimate RMU, we will model `d'` using a Bayesian random intercept and slope probit regression model. This approach will allow us to estimate uncertainty in each participant's fitted `d'` value, and calculate RMU reliability.

We will contrast code condition so that `condition_contrast = -1/2` for "no-go" trials, and `condition_contrast = +1/2` for "go" trials.

The random intercept captures individuals' propensity to generally respond "go" regardless of condition.

The random slope captures how well individuals shift their probability of responding "go" between conditions.

The fixed-effect (beta) coefficient for `condition_contrast` is the average difference, on the probit (z-score) scale, between the probability of responding "go" in the go and no-go conditions – in other words, the average `d'` across participants. Each participant's `d'` is this average plus their random slope.

Prepare Data for Modelling

Instead of modelling each individual trial, we will sum the number of "go" responses in each condition for each participant.

Because we are assuming that the probability of responding 'go' only varies by condition and participant, we do not need the individual trial data.

For example, if we wanted to estimate the probability of a flipped bent coin landing on heads, we just need to know the number of heads and number of throws, not the value of each individual coin toss.

Note that `correct` means different things in the two conditions: on go trials a correct response *is* a "go" response, whereas on no-go trials a correct response is *not* responding. So the number of "go" responses is `n_correct` in the go condition and `n - n_correct` in the no-go condition.

Finally, some participants have a perfect score in a condition (e.g. no false alarms at all on the no-go trials). The simple `d'` formula above cannot handle these, because `qnorm(0)` and `qnorm(1)` are infinite, so we nudge any count of `0` or `n` by one trial. The binomial model would cope without this adjustment, but we apply it here too so that the Bayesian and simple `d'` estimates compared in the appendix are calculated from identical data.

```
data_long_summarised = data_long |>
  group_by(pps, session, condition) |>
  summarise(
    n         = n(),
```

```
    n_correct = sum(correct),
    .groups = 'drop'
  ) |>
  ungroup() |>
  mutate(
    n_go = ifelse(condition == 0, n_correct, n - n_correct),
    condition_contrast = dplyr::case_when(
      condition == 0 ~ .5,
      condition == 2 ~ -.5
    )
  ) |>
  ungroup() |>
  mutate(
       n_go = ifelse(n_go == n, n_go - 1, n_go), # nudge perfect scores (see
text above)
      n_go = ifelse(n_go == 0, 1, n_go)
  ) |>
  arrange(pps)


data_long_summarised |>
  select(pps, session, condition_contrast, n, n_go) |>
  slice(1:10) |>
    knitr::kable(
      caption = "First 10 lines of our cleaned data for analysis",
      col.names = c("Unique Participant ID", "Testing Session",
                    "Condition (contrast coded)", "Number of Trials",
                    "Number of Go Responses in Condition")
    )
```

Unique Participant ID	Testing Session	Condition (contrast coded)	Number of Trials	Number of Go Responses in Condition
1	1	0.5	450	449
1	1	−0.5	150	62
1	2	0.5	450	448
1	2	−0.5	150	91
2	1	0.5	450	449
2	1	−0.5	150	41
2	2	0.5	450	449
2	2	−0.5	150	63

Unique Participant ID	Testing Session	Condition (contrast coded)	Number of Trials	Number of Go Responses in Condition
3	1	0.5	450	449
3	1	−0.5	150	32

Fit Bayesian Signal Detection Models

In the next code chunk, we fit our models to each session (time point) and we extract the posterior draws of each subject's `d'` value at each time point into two matrices called `subj_draws_time_1` and `subj_draws_time_2`.

I will stick with brms's default, weakly informative priors on the model parameters.

When initially running the models, we got a warning from Stan that "ESS is too low". We can address this by increasing the number of iterations (draws) per chain to 10,000 (setting `iter = 10000`). Note that half of these draws are discarded as warmup iterations. Setting `chains = 4` and `cores = 4` tells Stan to fit 4 chains and run them in parallel across four CPU cores, which will give 20,000 post-warmup draws in total.

For calculating test-retest reliability, it makes sense to fit models at each time point (session) independently.

```
model_time_1 = brms::brm(
  n_go | trials(n) ~  1 + condition_contrast + (1 + condition_contrast | pps),
  family = binomial(link = "probit"),
  data = filter(data_long_summarised, session == 1 ),
  cores   = 4,
  chains  = 4,
  iter    = 10000,
  seed    = 1 # So that the results are reproducible
)

model_time_2 = brms::brm(
  n_go | trials(n) ~  1 + condition_contrast + (1 + condition_contrast | pps),
  family = binomial(link = "probit"),
  data = filter(data_long_summarised, session == 2 ),
  cores   = 4,
  chains  = 4,
  iter    = 10000,
  seed    = 2
)

# Time Session 1
# Participant random effects of condition
```

```
subj_draws_time_1 = as_draws_df(model_time_1) |>
                    select(ends_with("condition_contrast]")) |>
                    t()

# Main effect of condition
cond_main_effect_1 = as_draws_df(model_time_1) |>
                     pull(b_condition_contrast) |>
                     as.numeric()

# Add main effect of condition to random effects.
# Each column of subj_draws_time_1 is one posterior draw, so we add the k-th
# main-effect draw to the k-th column (sweep over dimension 2 = columns).
# Note that `subj_draws_time_1 + cond_main_effect_1` would NOT do this: R would
# recycle the vector down the columns, pairing subjects with the wrong draws.
subj_draws_time_1 = sweep(subj_draws_time_1, 2, cond_main_effect_1, "+")


# Time / Session 2
# Participant random effects of condition
subj_draws_time_2 = as_draws_df(model_time_2) |>
                    select(ends_with("condition_contrast]")) |>
                    t()

# Main effect of condition
cond_main_effect_2 = as_draws_df(model_time_2) |>
                     pull(b_condition_contrast) |>
                     as.numeric()

# Add main effect of condition to random effects (see comment above)
subj_draws_time_2 = sweep(subj_draws_time_2, 2, cond_main_effect_2, "+")


# Calculate point-estimate of `d'` for each participant (posterior means)
# Time / Session 1
subj_est_time_1 = apply(subj_draws_time_1, 1, mean)
# Time / Session 2
subj_est_time_2 = apply(subj_draws_time_2, 1, mean)
```

Model results

```
model_time_1
```

```
 Family: binomial
  Links: mu = probit
Formula: n_go | trials(n) ~ 1 + condition_contrast + (1 + condition_contrast
| pps)
  Data: filter(data_long_summarised, session == 1) (Number of observations: 214)
  Draws: 4 chains, each with iter = 10000; warmup = 5000; thin = 1;
         total post-warmup draws = 20000

Multilevel Hyperparameters:
~pps (Number of levels: 107)
                                Estimate Est.Error l-95% CI u-95% CI Rhat
sd(Intercept)                       0.25      0.02     0.22     0.30 1.00
sd(condition_contrast)              0.79      0.06     0.68     0.92 1.00
cor(Intercept,condition_contrast)   0.14      0.11    -0.07     0.35 1.00
                                Bulk_ESS Tail_ESS
sd(Intercept)                       7129    12078
sd(condition_contrast)              5054     8745
cor(Intercept,condition_contrast)   2339     5381

Regression Coefficients:
                  Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept             0.78      0.03     0.73     0.83 1.00     7243    11294
condition_contrast    3.37      0.08     3.21     3.53 1.00     4258     7160

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
model_time_2
```

```
 Family: binomial
  Links: mu = probit
Formula: n_go | trials(n) ~ 1 + condition_contrast + (1 + condition_contrast
| pps)
  Data: filter(data_long_summarised, session == 2) (Number of observations: 214)
  Draws: 4 chains, each with iter = 10000; warmup = 5000; thin = 1;
         total post-warmup draws = 20000

Multilevel Hyperparameters:
~pps (Number of levels: 107)
                                Estimate Est.Error l-95% CI u-95% CI Rhat
sd(Intercept)                       0.26      0.02     0.22     0.31 1.00
```

```
sd(condition_contrast)                1.00      0.07     0.87     1.16 1.00
cor(Intercept,condition_contrast)     0.33      0.10     0.13     0.51 1.00
                                  Bulk_ESS Tail_ESS
sd(Intercept)                         4739     8992
sd(condition_contrast)                2687     6171
cor(Intercept,condition_contrast)     1363     2385

Regression Coefficients:
                  Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept             0.78      0.03     0.73     0.84 1.00     3456     6544
condition_contrast    3.14      0.10     2.94     3.33 1.01     1660     3490

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

Since random effects are centered (i.e., have a mean of zero), in the above code I have added the posterior distribution of the main effect of `condition_contrast` (which represents the average `d'` value across participants) to the random slope posteriors. This gives us posterior distributions that have the correct scaling for `d'`, but doesn't affect our reliability calculations.

At this point, the procedures for estimating RMU reliability are the same regardless of the model we have used to estimate participant effects.

For each of our models fitted to time 1 and time 2 data, we have a matrix with our posterior draws (`subj_draws_time_1` and `subj_draws_time_2`).

Each matrix has 107 rows (one row for each subject) and 20,000 columns (one column for each posterior draw). Each row has the posterior draws for a given subject's `d'`.

Below, we show data from 4 posterior draws from the first 10 participants.

Point estimates of `d'` for each participant could be estimated using the matrix row means (e.g., `rowMeans(subj_draws_time_1)`).

```
subj_draws_time_1 |>
  data.frame() |>
  slice(1:10) |>
  select(1:4)
```

```
                                  X1       X2       X3       X4
r_pps[1,condition_contrast]  2.767682 3.409310 2.594527 3.361499
r_pps[2,condition_contrast]  3.783342 2.714384 3.844913 2.896819
r_pps[3,condition_contrast]  3.546669 3.573018 3.402658 3.922859
r_pps[4,condition_contrast]  4.587050 4.554834 5.234301 4.149157
```

```
r_pps[5,condition_contrast]  2.818438 3.183356 2.611136 3.010641
r_pps[7,condition_contrast]  3.104741 2.949588 3.369073 3.590188
r_pps[8,condition_contrast]  3.625260 3.372115 3.731937 3.285019
r_pps[9,condition_contrast]  3.619500 3.860946 3.425238 3.913578
r_pps[10,condition_contrast] 2.976139 3.485962 2.745896 3.505298
r_pps[11,condition_contrast] 4.255406 4.320061 4.545379 4.485256
```

```
nrow(subj_draws_time_1) # number of subjects
```

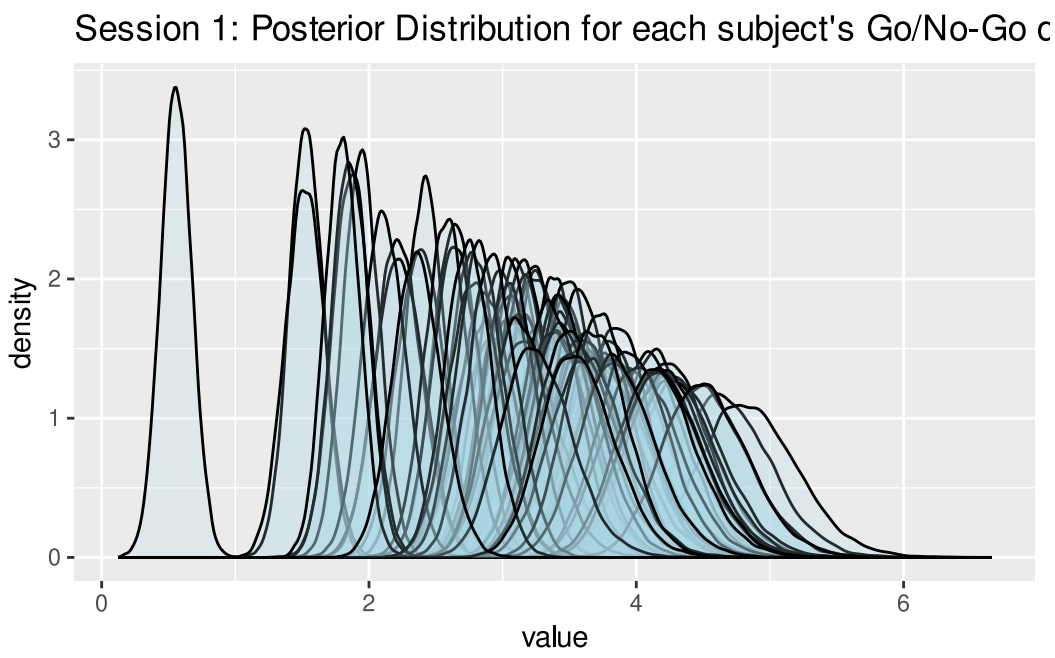
```
[1] 107
```

```
ncol(subj_draws_time_1) # number of posterior draws
```

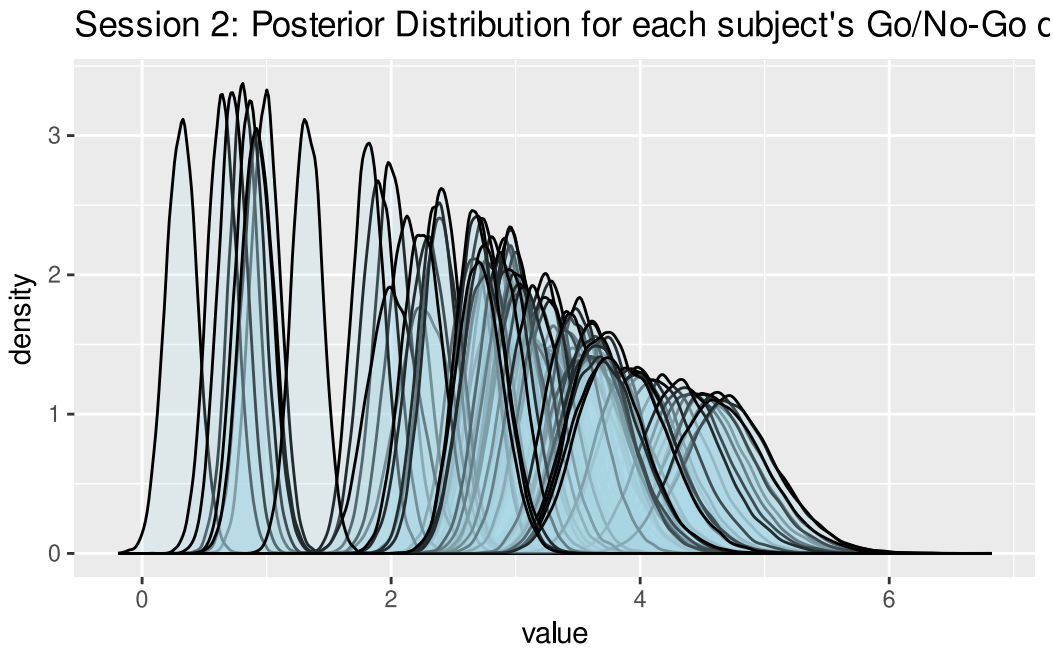
```
[1] 20000
```

To visualize our uncertainty about each subject's true d' value, we can plot the posterior distribution for each subject's `d'` value.

```
subj_draws_time_1 |>
  data.frame() |>
  rowid_to_column(var = "pps") |>
  pivot_longer(cols = !matches("pps")) |>
  ggplot(aes(x = value, group = pps)) +
  geom_density(alpha = .2, fill = "light blue") +
   labs(title = "Session 1: Posterior Distribution for each subject's Go/No-Go d' (sensitivity)")
```




```
subj_draws_time_2 |>
  data.frame() |>
  rowid_to_column(var = "pps") |>
  pivot_longer(cols = !matches("pps")) |>
  ggplot(aes(x = value, group = pps)) +
  geom_density(alpha = .2, fill = "light blue") +
   labs(title = "Session 2: Posterior Distribution for each subject's Go/No-
Go d' (sensitivity)")
```



From just eye-balling the posteriors, it is evident that there is a lot of variability between participants, and our estimates look quite precise. This suggests good reliability!

As expected, most of the mass of each density lies above 0. If `d'` is less than zero it would indicate that a participant is more likely to press the space bar (i.e., "go") for no-go conditions than go conditions, which would be odd.

Reliability Estimation Methods

In the following sections, we will outline a few different methods of calculating reliability and compare their results. We'll estimate RMU, test-retest, split-half, and empirical reliability.

RMU

Theory

RMU estimates reliability using the posterior distributions for each subject's true `d'` value.

To estimate RMU, `rmu()` first randomly splits the posterior draws into two halves (so with 20,000 posterior draws, each half contains 10,000). It then takes the first draw from each half, which gives every subject a pair of `d'` values, and calculates the correlation between the two across subjects. This correlation is a single "draw" of RMU reliability. Repeating this for every pair of draws (10,000 times here) builds up a posterior distribution for reliability, from which we can calculate point estimates and credible intervals.

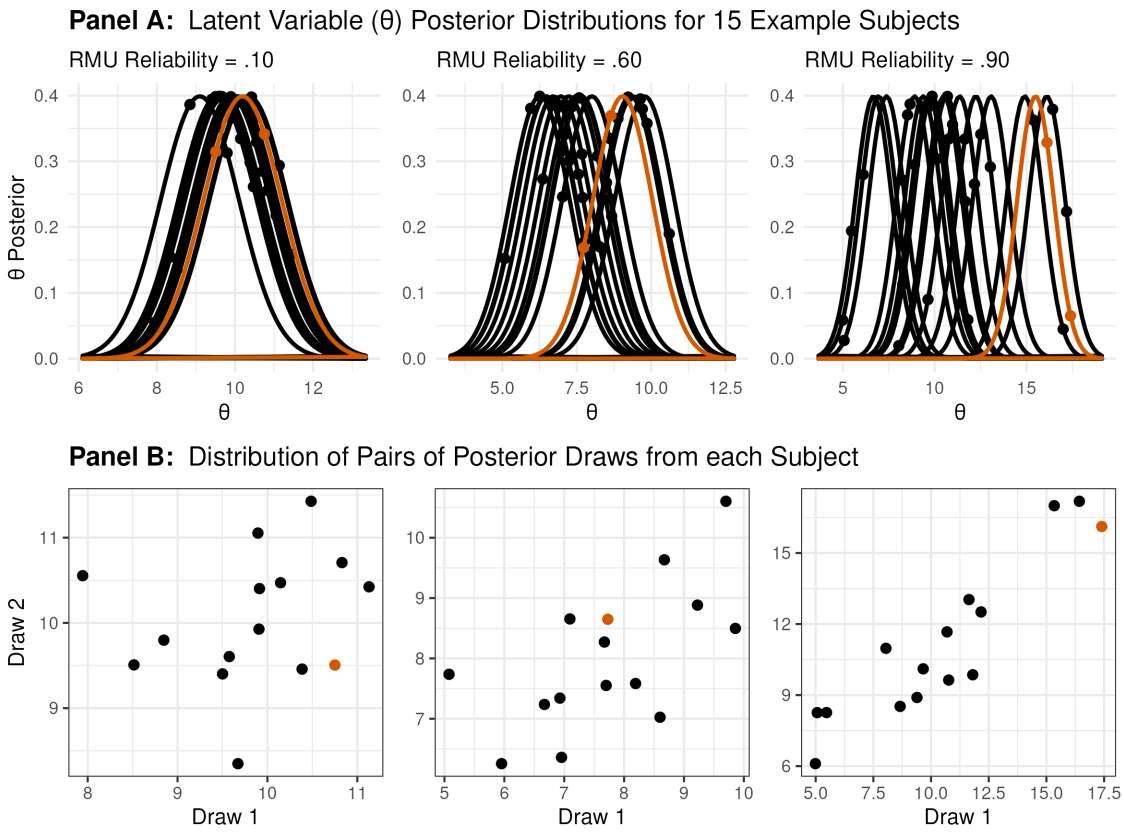


Figure 1: RMU reliability estimation process

One can think of RMU as a way of simulating test-retest reliability from a single session: each pair of draws is one simulated "test" and "retest" score per subject, and the correlation between them is one simulated test-retest correlation. RMU is closely related to the empirical reliability estimator, which estimates reliability by comparing the variance of the posterior means to the average posterior variance across all subjects.

RMU reliability will increase when subjects' posterior means have greater variance (more between-subject differences), or when subjects' posterior distributions become narrower (more precise individual estimates).

Practice

The code to calculate RMU using the posterior draw matrix is relatively simple. We use the `rmu()` function from `gbtoolbox` (installed and loaded in the Introduction).

Calculate RMU reliability at each time point. `rmu()` randomly splits the posterior draws in half, so we set a seed before each call to make the results reproducible (and to allow an exact comparison with the alternative approach shown further below):

```
set.seed(1)
rmu_time1 = rmu(subj_draws_time_1)

set.seed(1)
rmu_time2 = rmu(subj_draws_time_2)
```

RMU estimated using Time 1 data:

```
print(rmu_time1)
```

```
Reliability (Relative Measurement Uncertainty)
  Number of subjects:              107
  Number of input posterior draws: 20000
  Number of RMU posterior draws:   10000

  RMU estimate: 0.902
  Posterior SD: 0.014
  95% HDCI:     [0.874, 0.930]

Access full results with $hdci and $reliability_posterior_draws
```

RMU estimated using Time 2 data:

```
print(rmu_time2)
```

```
Reliability (Relative Measurement Uncertainty)
  Number of subjects:              107
  Number of input posterior draws: 20000
  Number of RMU posterior draws:   10000

  RMU estimate: 0.939
  Posterior SD: 0.009
  95% HDCI:     [0.921, 0.957]

Access full results with $hdci and $reliability_posterior_draws
```

The analyses indicate that reliability at the first time point was 90% (95% Credible Interval = 87%-93%), and at the second time point reliability was 94% (95% Credible Interval = 92%-96%).

Passing the fitted brms model directly

Instead of extracting the matrix of posterior draws yourself, you can pass the fitted `brms` model (an object of class `brmsfit`) straight to `rmu()`. It then extracts the posterior draws for every

group-level ("random") effect in the model and calculates RMU for each of them. Our model has two: the random intercept (each participant's general propensity to respond "go") and the random slope for `condition_contrast` (each participant's `d'`). The results are named `<grouping factor>__<coefficient>`, following the `sd_pps__Intercept`/`sd_pps__condition_contrast` naming that `brms` itself uses.

This method is new, so it currently prints a warning (once per R session) asking you to check its output carefully – which we do below:

```
set.seed(1)

rmu_time1_brms = rmu(model_time_1)
```

```
Warning: rmu() for brmsfit objects is new and has not yet been tested
extensively. Please check the output carefully and report any bugs at
https://github.com/giac01/gbtoolbox/issues (this warning is shown once per
session)
```

```

Reliability (Relative Measurement Uncertainty) Estimates
Posterior Mean & 95% HDCI
  Number of levels per grouping factor: pps = 107
  Number of input posterior draws:      20000
  Number of RMU posterior draws:        10000

                        rmu_estimate hdci_lowerbound hdci_upperbound
pps__Intercept                 0.789           0.728           0.844
pps__condition_contrast        0.903           0.874           0.930
```

`rmu()` prints a summary table and (invisibly) returns a list with one element per group-level effect, each with the same structure (`$hdci` and `$reliability_posterior_draws`) as the results above:

```
rmu_time1_brms$pps__condition_contrast$hdci
```

```
  rmu_estimate hdci_lowerbound hdci_upperbound .width .point .interval
1    0.9025173       0.8739341       0.9296698   0.95   mean      hdci
```

The `d'` (`pps__condition_contrast`) estimate is identical to `rmu_time1` above. The two approaches use exactly the same posterior draws (and we set the same seed, so `rmu()` split them in half in exactly the same way). Adding the main effect of `condition_contrast` to each draw, as we did when building `subj_draws_time_1`, shifts every participant's value within a draw by the same constant, so it has no effect on the correlations that RMU is built from.

In models with several grouping factors (e.g. participants and items), `rmu()` can't tell which one represents your subjects, so it returns results for all of them. Use the `group` and/or `coef` arguments to restrict the output to the effects you're interested in:

```
set.seed(1)

rmu_time2_brms = rmu(model_time_2, group = "pps", coef = "condition_contrast")
```

```
Reliability (Relative Measurement Uncertainty) Estimates
Posterior Mean & 95% HDCI
  Number of levels per grouping factor: pps = 107
  Number of input posterior draws:      20000
  Number of RMU posterior draws:        10000

                        rmu_estimate hdci_lowerbound hdci_upperbound
pps__condition_contrast        0.939           0.921           0.957
```

Test-Retest Reliability

The test-retest reliability is a little lower than these estimates (r = 0.73):

```
cor.test(subj_est_time_1, subj_est_time_2)
```

```
    Pearson's product-moment correlation

data:  subj_est_time_1 and subj_est_time_2
t = 10.876, df = 105, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.6242545 0.8062874
sample estimates:
      cor
0.7278483
```

If there were no changes in individuals' latent (true) `d'` values across time points, the population test-retest reliability and RMU would be identical. Because some change over time is inevitable, test-retest reliability will usually be a bit lower than reliability estimates derived from a single time point.

Our estimate for split-half reliability (see below) is similar to RMU, because it's not affected by change in participants' performance over time.

Calculating a posterior for test-retest reliability

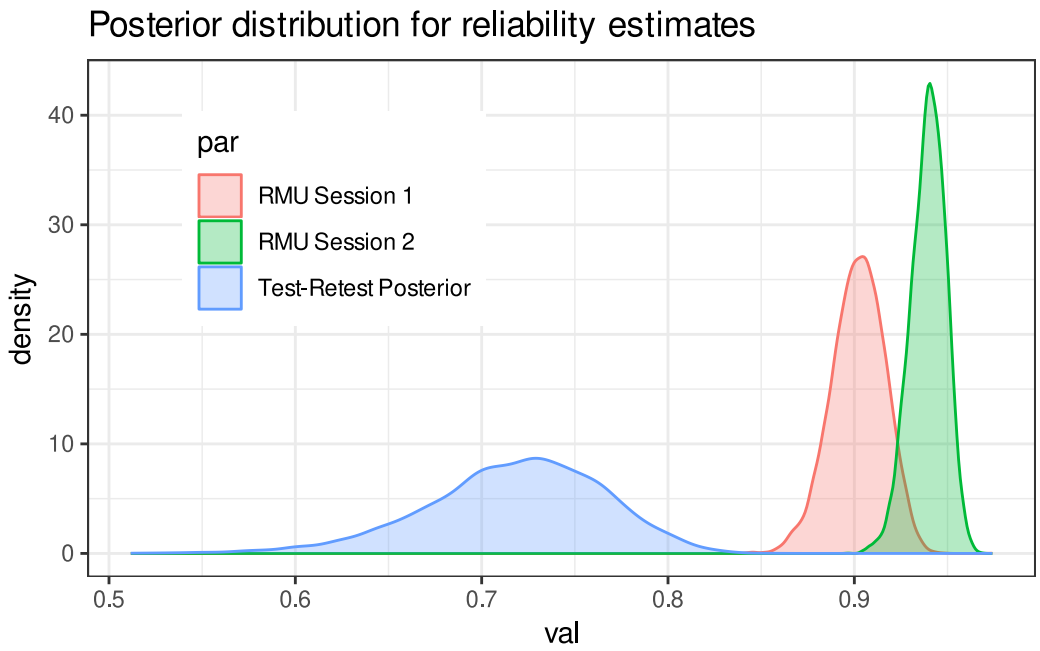
While `cor.test` gives us a confidence interval, if we want a posterior distribution for this correlation we can use the following code thanks to Solomon Kurz for code.

```
test_retest_cor = data.frame(subj_est_time_1, subj_est_time_2) |>
  brms::brm(
       bf(mvbind(subj_est_time_2,subj_est_time_1 ) ~ 1,
         sigma ~ 1) + set_rescor(TRUE),
       data = _,
       chains = 6,
       cores = 6,
       seed = 1,
       control = list(adapt_delta = .95)
       )

test_retest_cor_posterior = test_retest_cor |>
  tidybayes::spread_draws(rescor__subjesttime2__subjesttime1)
```

We can then visually compare test-retest reliability with our RMU estimates:

```
bind_rows(
          data.frame(par     =     "Test-Retest     Posterior",     val     =
test_retest_cor_posterior$rescor__subjesttime2__subjesttime1),
 data.frame(par = "RMU Session 1", val = rmu_time1$reliability_posterior_draws),
 data.frame(par = "RMU Session 2", val = rmu_time2$reliability_posterior_draws)
) |>
  ggplot(aes(x = val, group = par, col = par, fill = par)) +
  geom_density(alpha = .3) +
  theme_bw() +
  theme(
    legend.position = c(.1, .9),
    legend.justification = c(0, 1)
  ) +
  labs(title = "Posterior distribution for reliability estimates")
```



Split-Half Reliability

If we want to avoid model-fitting, another way of calculating reliability is to use split-half reliability.

To calculate split-half reliability, we split each participant's trials into odd- and even-numbered trials, calculate `d'` for each half (at each time point) using the simple hit rate and false alarm rate formula from Equation 1, and correlate the two halves.

These correlations are then transformed using the Spearman-Brown "prophecy" formula, which corrects for the fact that each half only contains half the trials.

The results are quite similar, though the split-half estimates are slightly higher.

The function below does the summarising and `d'` calculation for one half of the trials; it is the same data preparation as above (including the nudging of perfect scores), followed by the `d'` formula:

```
split_half_dprime = function(data_long, remainder){
  data_long |>
    filter(trial %% 2 == remainder) |> # 1 = odd trials, 0 = even trials
    group_by(pps, session, condition) |>
    summarise(
      n         = n(),
      n_correct = sum(correct),
      .groups   = "drop"
    ) |>
```

```
    mutate(
      n_go      = ifelse(condition == 0, n_correct, n - n_correct),
      n_go      = ifelse(n_go == n, n_go - 1, n_go),
      n_go      = ifelse(n_go == 0, 1, n_go),
      condition = factor(ifelse(condition == 0, "go", "nogo"))
    ) |>
    arrange(pps) |>
    pivot_wider(
      id_cols = pps,
      names_from = c(condition, session),
      values_from = c(n, n_go),
      names_glue = "{condition}_t{session}_{.value}"
    ) |>
    mutate(
      hit_rate_t1        = go_t1_n_go / go_t1_n,
      falsealarm_rate_t1 = nogo_t1_n_go / nogo_t1_n,
      hit_rate_t2        = go_t2_n_go / go_t2_n,
      falsealarm_rate_t2 = nogo_t2_n_go / nogo_t2_n,
      d_prime_t1         = qnorm(hit_rate_t1) - qnorm(falsealarm_rate_t1),
      d_prime_t2         = qnorm(hit_rate_t2) - qnorm(falsealarm_rate_t2)
    )
}

data_long_odd  = split_half_dprime(data_long, remainder = 1)
data_long_even = split_half_dprime(data_long, remainder = 0)

split_half_cor_t1 = cor(data_long_odd$d_prime_t1, data_long_even$d_prime_t1)
split_half_cor_t2 = cor(data_long_odd$d_prime_t2, data_long_even$d_prime_t2)

data.frame(
  par = c("Split Half Correlation (T1)", "Split Half Correlation (T2)", "Split-
Half Reliability", "Split-Half Reliability"),
  val = c(
    split_half_cor_t1,
    split_half_cor_t2,
    (2*split_half_cor_t1)/(1+split_half_cor_t1),
    (2*split_half_cor_t2)/(1+split_half_cor_t2))
) |>
  knitr::kable(
    digits = 4,
      caption = "Split-Half Correlations and Spearman-Brown Corrected Split-
Half Reliabilities"
    )
```

par	val
Split Half Correlation (T1)	0.8944
Split Half Correlation (T2)	0.9170
Split-Half Reliability	0.9442
Split-Half Reliability	0.9567

Empirical Reliability

Finally, the empirical reliability method involves calculating the variance of the posterior means ($\hat{\boldsymbol{\theta}}$) and average posterior variance across subjects ($\hat{\boldsymbol{V}}_i$).

Empirical reliability can also be used with maximum likelihood estimates of `d'` and error variances - which can make it appealing to those less familiar with Bayesian methods. Instead of brms/stan, you could use lme4 or other packages to fit the signal detection theory model using maximum likelihood. However, I'm not aware of any methods to get uncertainty intervals with empirical reliability, beyond perhaps bootstrapping.

$$\text{Empirical Reliability} = \frac{\text{Var}\ (\hat{\boldsymbol{\theta}})}{\text{Var}\ (\hat{\boldsymbol{\theta}}) + \sum_{i=1}^{N} \hat{\boldsymbol{V}}_i / N} \tag{2}$$

```
variance_posterior_means_t1 = var(apply(subj_draws_time_1, 1, mean))
variance_posterior_means_t2 = var(apply(subj_draws_time_2, 1, mean))
average_posterior_variance_t1 = mean(apply(subj_draws_time_1, 1, var))
average_posterior_variance_t2 = mean(apply(subj_draws_time_2, 1, var))

empirical_reliability_t1        =        variance_posterior_means_t1        /
(variance_posterior_means_t1 + average_posterior_variance_t1)

empirical_reliability_t2        =        variance_posterior_means_t2        /
(variance_posterior_means_t2 + average_posterior_variance_t2)

data.frame(
  Method = c("RMU Session 1", "RMU Session 2", "Empirical Session 1", "Empirical
Session 2"),
  Reliability = c(
    rmu_time1$hdci$rmu_estimate,
    rmu_time2$hdci$rmu_estimate,
    empirical_reliability_t1,
    empirical_reliability_t2
  )
) |>
  knitr::kable(
```

```
    digits = 4,
    caption = "Comparison of RMU and Empirical Reliability Point Estimates"
    )
```

Method	Reliability
RMU Session 1	0.9020
RMU Session 2	0.9395
Empirical Session 1	0.9010
Empirical Session 2	0.9387

Appendix: Bayesian vs. Simple d' Estimates

If we just calculate `d'` using the transformed hit rate and false alarm rate, using the formula shown in Equation 1, does this give us estimates that are very different, or less reliable?

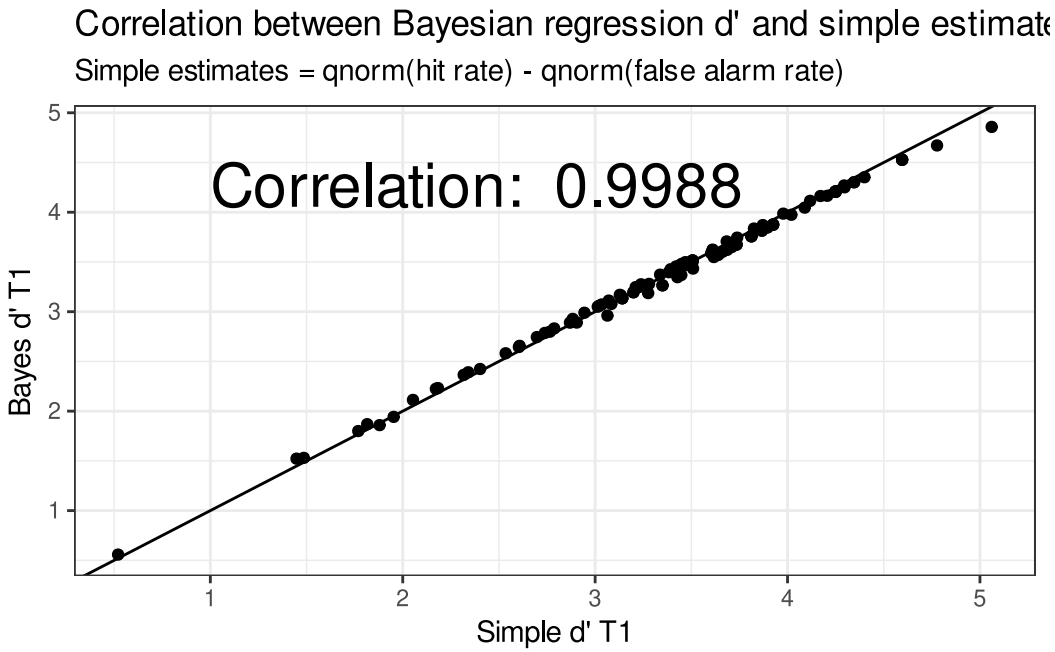
The answer in this case seems to be no, the estimates are very similar regardless of which approach we take:

```
two_stage_estimate =
 data_long_summarised |>
  select(-condition_contrast) |>
  mutate(condition = factor(ifelse(condition==0,"go","nogo"))) |>
  pivot_wider(
    id_cols = pps,
    names_from = c(condition, session),
    values_from = c(n, n_go),
    names_glue = "{condition}_t{session}_{.value}"
              ) |>
  mutate(
    hit_rate_t1        = go_t1_n_go/(go_t1_n),
    falsealarm_rate_t1 = nogo_t1_n_go/nogo_t1_n,
    hit_rate_t2        = go_t2_n_go/go_t2_n,
    falsealarm_rate_t2 = nogo_t2_n_go/nogo_t2_n,
    d_prime_t1         = qnorm(hit_rate_t1) - qnorm(falsealarm_rate_t1),
    d_prime_t2         = qnorm(hit_rate_t2) - qnorm(falsealarm_rate_t2)
  )

data_plot = data.frame(
  `Simple d' T1` = two_stage_estimate$d_prime_t1,
  `Simple d' T2` = two_stage_estimate$d_prime_t2,
  `Bayes d' T1` = subj_est_time_1,
  `Bayes d' T2` = subj_est_time_2
```

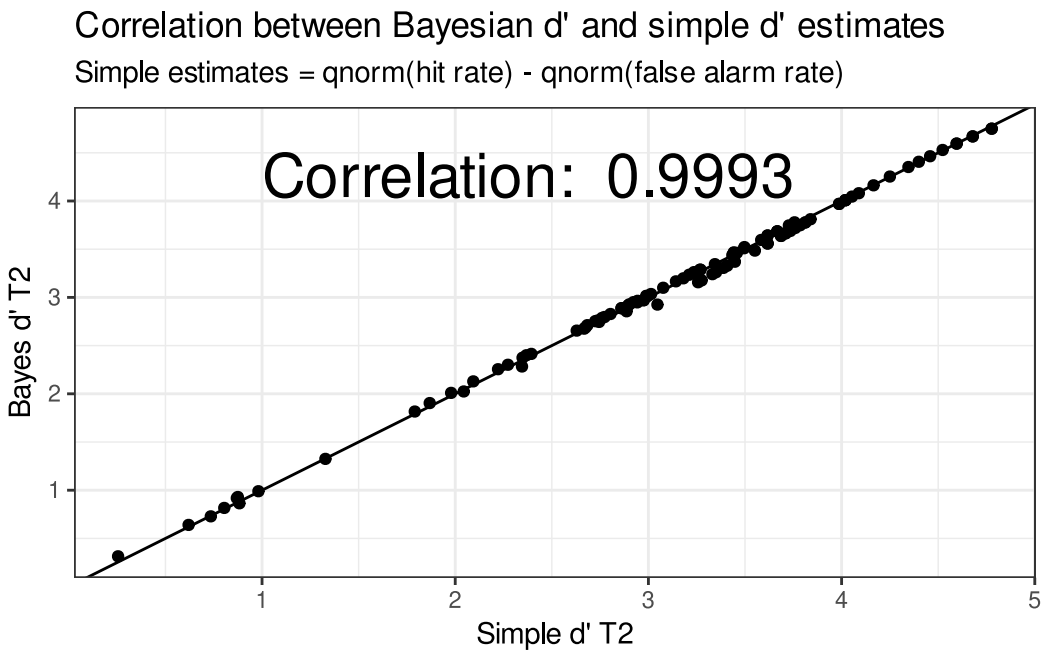
```
)

data_plot |>
  ggplot(aes(x = Simple.d..T1, Bayes.d..T1)) +
  geom_abline(slope = 1, intercept = 0) +
  geom_point() +
  theme_bw() +
  annotate("text", x = 1, y = 4.5,
                                        label    =    paste("Correlation:    ",
round(cor(two_stage_estimate$d_prime_t1,subj_est_time_1), 4)),
           hjust = 0, vjust = 1, size = 8) +
  labs(
    x = "Simple d' T1",
    y = "Bayes d' T1",
    title = "Correlation between Bayesian regression d' and simple estimates",
    subtitle = "Simple estimates = qnorm(hit rate) - qnorm(false alarm rate)"
  )
```




```
data_plot |>
  ggplot(aes(x = Simple.d..T2, Bayes.d..T2)) +
  geom_abline(slope = 1, intercept = 0) +
  geom_point() +
  theme_bw() +
```

```
  annotate("text", x = 1, y = 4.5,
                                        label    =    paste("Correlation:    ",
round(cor(two_stage_estimate$d_prime_t2,subj_est_time_2), 4)),
           hjust = 0, vjust = 1, size = 8) +
  labs(
    x = "Simple d' T2",
    y = "Bayes d' T2",
    title = "Correlation between Bayesian d' and simple d' estimates",
    subtitle = "Simple estimates = qnorm(hit rate) - qnorm(false alarm rate)"
  )
```



This example highlights that there isn't anything magical about Bayesian or more fancy statistical methods. They don't provide better estimates by default.

This doesn't mean that the regression approach to calculating `d'` is pointless. As we have shown, we can easily calculate reliability from our Bayesian regression model - even without retest data. We could also calculate a credible interval for any participant's true `d'` score. We can also handle more complex signal detection theory models within a regression approach, or we could control for time of day or practice effects, and more.

Session Info

```
sessionInfo()
```

```
R version 4.6.1 (2026-06-24)
Platform: x86_64-pc-linux-gnu
Running under: Ubuntu 24.04.4 LTS

Matrix products: default
BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
LAPACK:    /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblasp-r0.3.26.so;
LAPACK version 3.12.0

locale:
 [1] LC_CTYPE=en_US.UTF-8       LC_NUMERIC=C
 [3] LC_TIME=en_US.UTF-8        LC_COLLATE=en_US.UTF-8
 [5] LC_MONETARY=en_US.UTF-8    LC_MESSAGES=en_US.UTF-8
 [7] LC_PAPER=en_US.UTF-8       LC_NAME=C
 [9] LC_ADDRESS=C               LC_TELEPHONE=C
[11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C

time zone: Etc/UTC
tzcode source: system (glibc)

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
 [1] gbtoolbox_0.0.4 gt_1.3.0        brms_2.23.0     Rcpp_1.1.2
 [5] tidybayes_3.0.7 lubridate_1.9.5 forcats_1.0.1   stringr_1.6.0
 [9] dplyr_1.2.1     purrr_1.2.2     readr_2.2.0     tidyr_1.3.2
[13] tibble_3.3.1    ggplot2_4.0.3   tidyverse_2.0.0

loaded via a namespace (and not attached):
 [1] svUnit_1.0.8          tidyselect_1.2.1      farver_2.1.2
 [4] loo_2.10.0            S7_0.2.2              fastmap_1.2.0
 [7] tensorA_0.36.2.1      digest_0.6.39         timechange_0.4.0
[10] lifecycle_1.0.5       StanHeaders_2.32.10   processx_3.9.0
[13] magrittr_2.0.5        posterior_1.7.0       compiler_4.6.1
[16] rlang_1.3.0           sass_0.4.10           tools_4.6.1
[19] yaml_2.3.12           knitr_1.51            labeling_0.4.3
[22] bridgesampling_1.2-1  pkgbuild_1.4.8        curl_7.1.0
[25] plyr_1.8.9            xml2_1.6.0            RColorBrewer_1.1-3
[28] abind_1.4-8           withr_3.0.3           grid_4.6.1
[31] stats4_4.6.1          inline_0.3.21         scales_1.4.0
[34] cli_3.6.6             mvtnorm_1.4-1         rmarkdown_2.31
[37] generics_0.1.4        remotes_2.5.0         otel_0.2.0
```

```
[40] RcppParallel_5.1.11-2 reshape2_1.4.5        tzdb_0.5.0
[43] rstan_2.32.7          bayesplot_1.15.0      parallel_4.6.1
[46] matrixStats_1.5.0     vctrs_0.7.3           V8_8.2.0
[49] Matrix_1.7-5          jsonlite_2.0.0        callr_3.8.0
[52] hms_1.1.4             arrayhelpers_1.1-1    ggdist_3.3.3
[55] glue_1.8.1            codetools_0.2-20      ps_1.9.3
[58] distributional_0.8.1  stringi_1.8.7         gtable_0.3.6
[61] QuickJSR_1.10.0       pillar_1.11.1         htmltools_0.5.9
[64] Brobdingnag_1.2-9     R6_2.6.1              evaluate_1.0.5
[67] lattice_0.22-9        backports_1.5.1       rstantools_2.6.0
[70] coda_0.19-4.1         gridExtra_2.3.1       nlme_3.1-169
[73] checkmate_2.3.4       xfun_0.59             fs_2.1.0
[76] pkgconfig_2.0.3
```